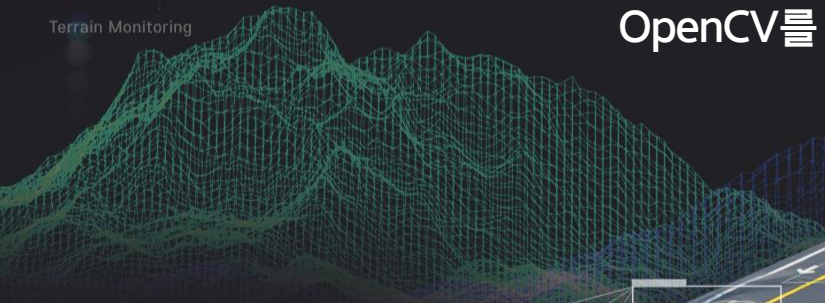


Image Recognition



Terrain Monitoring



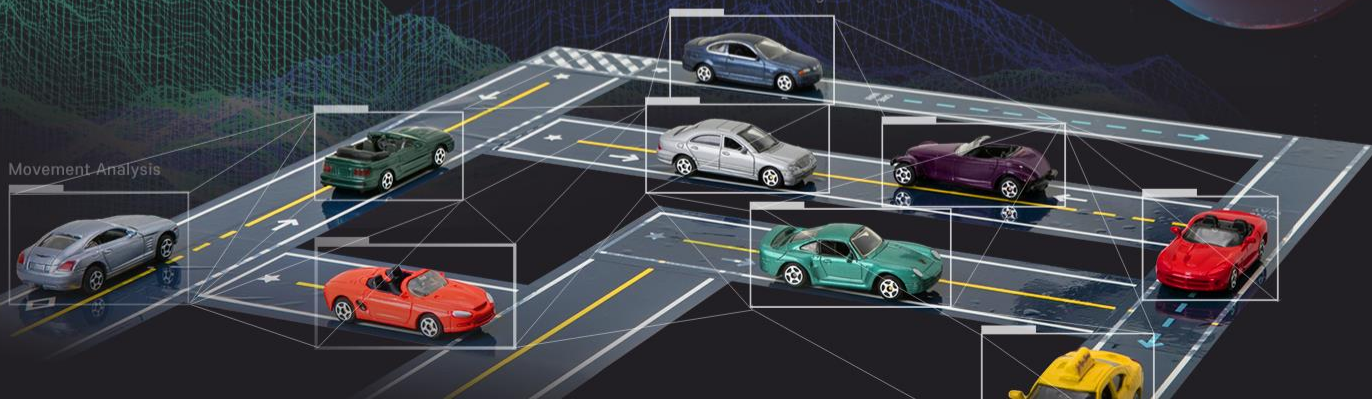
강원혁신플랫폼

컴퓨터 비전

OpenCV를 활용한 영상 정보 전처리(2)

Autonomous Driving

Movement Analysis



이미지에 그리기

4.openCVDRAW.ipynb : 이미지에 선, 사각형, 원, Text 그리기

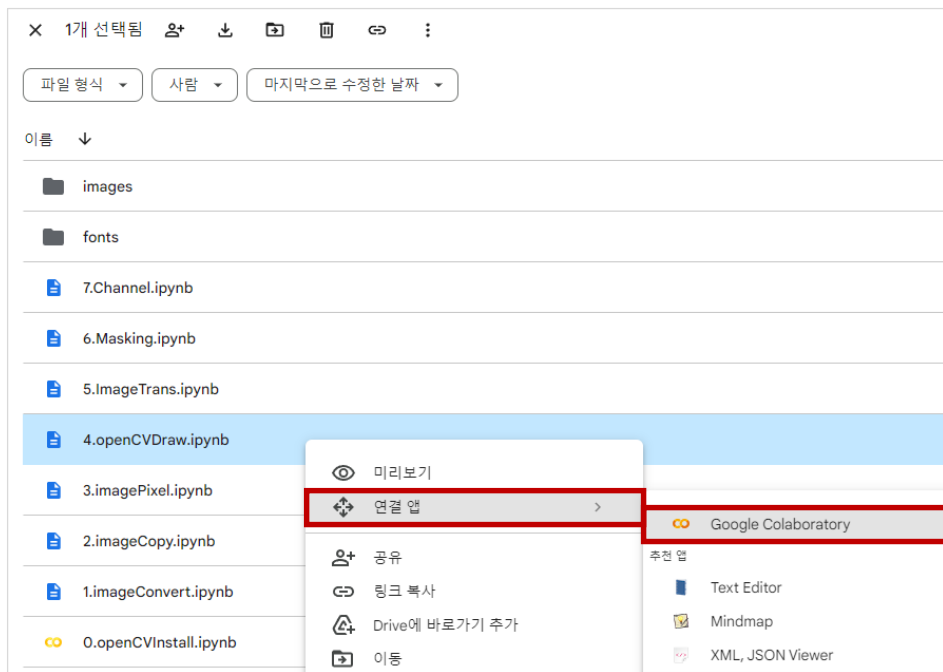
OpenCV를 이용해 아래의 기능을 수행해 보자.

이미지에 선, 사각형, 원을 그린다.

이미지에 Text를 출력한다.

1 openCVDDraw.ipynb 구글 Colab 연동

- 아래의 그림의 Basic Vision Skills 폴더에서 4.openCVDDraw.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭한다.



1 openCVDRAW.ipynb 구글 Colab 연동

■ 아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



2 OpenCV 버전 확인

■ openCV를 사용하기 위해 필요한 모듈을 읽고, openCV 버전을 확인해 보자.

🔄 openCV를 사용하기 위해 **cv2 패키지**를 읽어온다.

```
1 # 필요한 모듈 읽어오기
2
3 import cv2
4 from google.colab.patches import cv2_imshow
```

🔄 openCV를 설치 **버전**을 **확인**한다.

```
1 # OpenCV 버전을 확인합니다
2
3 print("OpenCV version:")
4 print(cv2.__version__)
```

```
OpenCV version:
4.7.0
```

3 구글 Colab 화면에서 내 구글 드라이브 연동

■ 내 구글 드라이브에 연동한다.

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# 내 구글 드라이브에 연동
from google.colab import drive

drive.mount('/content/gdrive')
```

Mounted at /content/gdrive

- [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동된다.
- gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.

4 구글 Colab 화면에서 내 구글 드라이브 연동

[Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용]

아래 화면을 참고하여 gdrive에 연동한다.

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

[아니오](#) [Google Drive에 연결](#)

Google 계정으로 로그인

계정 선택

Google Drive for desktop(으)로 이동

[장원중](#)
wjang2040@gmail.com

다른 계정 사용

계속 진행하기 위해 Google에서 내 이름, 이메일 주소, 언어 환경 설정, 프로필 사진을 Google Drive for desktop(와) 공유합니다. 앱을 사용하기 전에 Google Drive for desktop의 [개인정보처리방침](#) 및 서비스 약관을 검토하세요.

한국어 | 도움말 | 개인정보처리방침 | 약관

구글 드라이브 접근 허용

Google 계정으로 로그인

Google Drive for desktop에서 내 Google 계정에 액세스하려고 합니다.

[wjang2040@gmail.com](#)

어떻게 하면 Google Drive for desktop에서 내 작업을 할 수 있습니까?

- Google Drive 파일 보기, 수정, 생성, 삭제
- Google 포토의 사진, 동영상, 앨범을 봅니다.
- 모바일 클라이언트 구성 및 실행 정보 가져오기
- 프로필 및 연락처와 같은 Google 사용자 정보 조회
- Google Drive 파일 작업 기록 조회
- Google Drive 문서 보기, 수정, 생성, 삭제

Google Drive for desktop 앱을 신뢰할 수 있는지 확인

(제한된 정보가 이 사이트 또는 앱과 공유될 수 있습니다. 언제든지 Google 계정에서 액세스 권한을 확인하고 삭제할 수 있습니다.)

Google이 데이터를 안전하게 공유하는 방법을 알아보세요.

Google Drive for desktop이 개인정보처리방침 및 서비스 약관을 확인하세요.

[취소](#) [허용](#)

한국어 | 도움말 | 개인정보처리방침 | 약관

5 원본 이미지 읽기 → 형상(세로, 가로, 채널)

■ 다음은 cv2.imread() 함수로 원본 이미지를 읽어온다.

🔄 원본 이미지를 읽어올 때 컬러로 읽어온다.

```
1 # 이미지를 읽어 온다.  
2  
3 img = cv2.imread('gdrive/My Drive/CV/Basic Vision Skills/images/rear_garden.PNG', cv2.IMREAD_COLOR)  
4 print(img.shape) # 형상 - (496, 819, 3)  
5 print("width: {} pixels".format(img.shape[1])) # 넓이  
6 print("height: {} pixels".format(img.shape[0])) # 높이  
7 print("channels: {}".format(img.shape[2])) # 채널
```

```
📄 (496, 819, 3)  
width: 819 pixels  
height: 496 pixels  
channels: 3
```

위 결과에서 원본 이미지의 형상은 (496, 819, 3)인 것을 알 수 있다.

5 원본 이미지 읽기 → 형상(세로, 가로, 채널)

■ 다음은 cv2_imshow() 함수로 원본 이미지를 화면에 출력한다.

🔄 원본 이미지(PNG 형식)를 화면에 출력한다.

```
1 # 원본 이미지를 화면에 출력  
2 cv2_imshow(img)
```



6 원본 이미지의 특정한 범위 빨간 색상으로 변경

■ 원본 이미지의 [0, 1] 위치는 어떠한 Color 정보를 가지고 있는지 알아보자.

🔄 원본 이미지의 색상을 정할 때 OpenCV 이미지의 기본 채널 순서는 파랑-녹색-빨간(BGR) 방식을 사용한다.

```
1 # 색상을 정할 때 OpenCV 이미지의 기본 채널 순서는 파랑-녹색-빨간(BGR) 방식을 사용한다.  
2  
3 # 이미지의 특정 위치 = 픽셀(Pixel), Pixel(BGR)  
4 (b, g, r) = img[0, 1]  
5 print("Pixel at (0, 0) - Red: {}, Green: {}, Blue: {}".format(r, g, b))
```

👉 Pixel at (0, 0) - Red: 12, Green: 20, Blue: 5

[0, 1] 위치의 픽셀 값은 R=12, G=20, B=5 인 색상 정보를 가지고 있음을 확인 할 수 있다.

6 원본 이미지의 특정한 범위 빨간 색상으로 변경

■ 원본 이미지의 특정 범위 [50:100, 50:100] 영역에 파란색으로 변경해 이미지를 출력해 보자.

🔄 원본 이미지의 [50:100, 50:100] 영역에 (255,0,0) 픽셀값을 설정한다.

```
1 # 이미지 내에서 특정한 범위만 지정하여 파란 색상으로 변경한다.  
2 img[50:100, 50:100] = (255, 0, 0)  
3 cv2.imshow(img) # 변경 이미지 출력
```

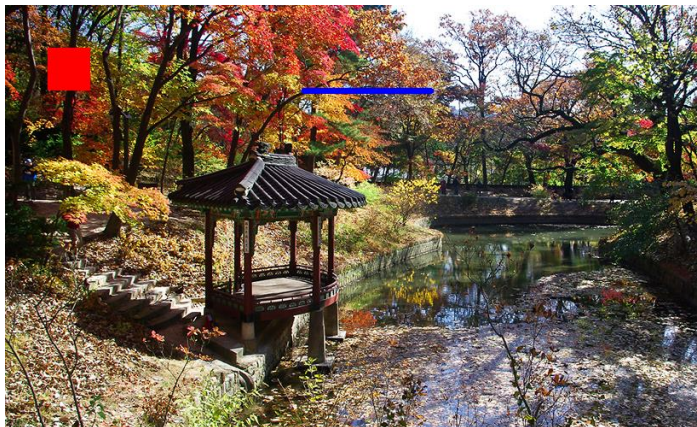


7 원본 이미지에 선 그리기

■ 원본 이미지의 특정한 위치에 선을 추가하고 화면에 출력한다.

🔄 cv2.line(이미지, 시작지점 좌표, 끝지점 좌표, 색상, 굵기, 선의 종류, 좌표 시프트) 함수는 이미지에 선을 그린다.

```
1 # cv2.line(이미지, 시작지점 좌표, 끝지점 좌표, 색상, 굵기, 선의 종류, 좌표 시프트)
2
3 # 이미지에 선을 그린다.
4 cv2.line(img, (350, 100), (500, 100), (255, 0, 0), 5) # (원본이미지, 시작점, 끝점, 색상, 선굵기)
5 cv2.imshow(img) # 이미지 출력
```

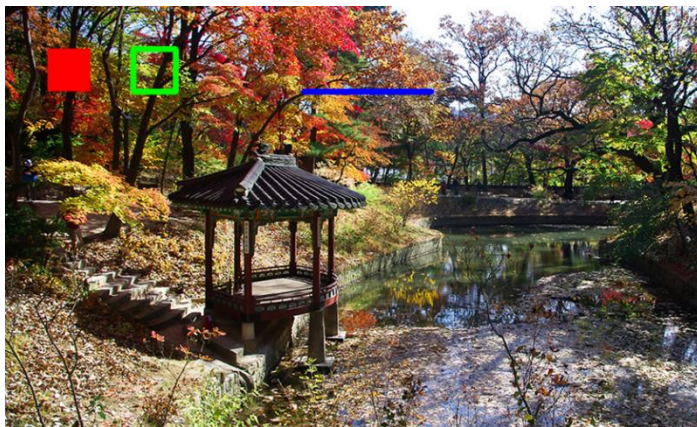


8 원본 이미지에 사각형 그리기

■ 원본 이미지의 특정한 위치에 사각형을 추가하고 화면에 출력한다.

🔄 cv2.rectangle(이미지, 시작지점 좌표, 끝지점 좌표, 색상, 선 두께, 선의 종류, 좌표 시프트) 함수는 이미지에 사각형을 그린다.

```
1 # cv2.rectangle(이미지, 시작점 좌표, 종료점 좌표, 색상, 선 두께, 선 종류, 좌표 시프트)
2
3 # 이미지에 사각형을 그린다.
4 cv2.rectangle(img, (150, 50), (200, 100), (0, 255, 0), 5) # (원본이미지, 시작점 좌표, 끝점 좌표, 색상, 선굵기)
5 cv2.imshow('img') # 이미지 출력
```



9 원본 이미지에 원 그리기

■ 원본 이미지의 특정한 위치에 원을 추가하고 화면에 출력한다.

🔄 cv2.circle(이미지, 중심점, 반지름, 색상, -1은 도형전체를 채움) 함수는 이미지에 원을 그린다.

```
1 # cv2.circle()
2
3 # 이미지에 원을 그린다.
4 cv2.circle(img, (275, 75), 25, (0, 255, 255), -1) # (원본 이미지, 중심점, 반지름, 색상, -1은 도형전체를 채움)
5 cv2.imshow(img) # 이미지 출력
```

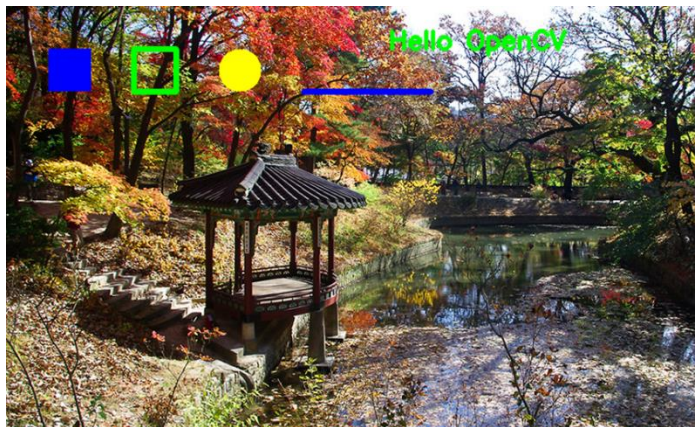


9 원본 이미지에 원 그리기

■ 원본 이미지의 특정한 위치에 글자를 추가하고 화면에 출력한다.

🔄 cv2.putText(이미지, 입력글, 시작점, 폰트, 폰트크기, 폰트색상, 폰트굵기) 함수는 이미지에 글자를 그린다.

```
1 # cv2.putText(원본 이미지, 입력글, 시작점, 폰트, 폰트크기, 폰트색상, 폰트굵기)
2
3 # 이미지에 TEXT를 씁니다.
4 cv2.putText(img, 'Hello OpenCV', (450, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 4) # (원본 이미지, 입력글, 시작점,
5 cv2.imshow(img) # 이미지 출력
```



10 이미지에 한글 추가하기

■ 원본 이미지의 특정한 위치에 한글을 추가하고 화면에 출력한다.

🔄 cv2.putText() 함수를 이용해 Text를 쓸 때 영문은 출력에 문제없지만 한글은 글자가 깨지는 현상이 발생한다.



따라서, 한글 출력은 PIL(Python Imaging Library) 모듈을 이용해 출력할 수 있다.

🔄 한글 출력에 필요한 모듈(패키지, 라이브러리)을 불러온다.



필요한 라이브러리를 불러오기

```
import numpy as np
from PIL import ImageFont, ImageDraw, Image
```

11 원본 이미지 형식 변경(BGR → RGBA)

■ 이미지에 한글을 출력하고, 폰트에 투명도를 적용시켜 화면에 출력한다.

⦿ 앞의 이미지는 cv2.imread() 함수로 불러왔기 때문에 BGR 형식으로 되어 있어 폰트에 투명도를 나타낼 수 없다.

➤ 따라서, RGBA 형식(투명포함 4채널)으로 변경해야 한다(A는 alpha 값 투명도를 나타낸다).

⦿ 다음은 BGR 이미지 형식을 RGBA 형식으로 변환한다.

```
1 # BGR to RGBA 이미지 변환
2
3 print(img.size)           # size = 1218672 = 496 * 819 * 3
4 print(img.shape)          # 형상 = (496, 819, 3)
5 base_img = Image.fromarray(img).convert('RGBA') # BGR ==> RGBA 변환
6 print(base_img.size)      # (819, 496)
```

```
1218672
(496, 819, 3)
(819, 496)
```


12 한글 폰트 설정, Blank 이미지에 한글 출력

- 다음은 사용할 한글 폰트를 설정한다.

```
fontpath = "gdrive/My Drive/CV/Basic Vision Skills/fonts/HYTBRB.TTF" # 폰트 경로
font = ImageFont.truetype(fontpath, 70) # 폰트, 폰트크기 지정
```

- 텍스트를 작성할 Blank 이미지를 하나 만들고, Blank 이미지에 한글을 출력한다.

- 투명도는 완전 투명, 반투명, 불투명으로 설정한다.

```
# 텍스트를 작성할 blank 이미지를 하나 만든다.
txt_img = Image.new('RGBA', base_img.size, (255,255,255,0))

draw = ImageDraw.Draw(txt_img) # blank 이미지에 텍스트를 넣기 위해 draw 인스턴스를 만든다
draw.text((200, 150), "컴퓨터비전", font=font, fill=(255,255,255,0)) # alpha 완전투명
draw.text((200, 250), "컴퓨터비전", font=font, fill=(255,255,255,128)) # alpha 반투명
draw.text((200, 350), "컴퓨터비전", font=font, fill=(255,255,255,255)) # alpha 불투명
```

13 원본 이미지 + Blank 이미지 결합 → 화면 출력

■ 원본 이미지(base_img)와 Blank 이미지(txt_img)를 합치고, 합친 이미지를 화면에 출력한다.

```
1 comp_img = Image.alpha_composite(base_img,txt_img)           # base_img 와 txt_img 을 합친다.  
2 print(type(comp_img))                                         # <class 'PIL.Image.Image'>  
3 img = np.array(comp_img)                                       # 배열 자료형으로 변환  
4 print(type(img))                                              # <class 'numpy.ndarray'>  
5 print(img.shape)                                              # 형상 - (496, 819, 4)  
6 cv2_imshow(img)                                              # 이미지 출력
```

```
> <class 'PIL.Image.Image'>  
> <class 'numpy.ndarray'>  
> (496, 819, 4)
```



원본 이미지 + Blank 이미지 결합해 출력

아래 그림에서 투명도를 0으로 설정된 한글은 완전 투명도가 적용되어 보이지 않는다.
투명도가 반투명도, 불투명도가 적용된 한글은 보여지고 있음을 확인 할 수 있다.



OpenCV 그리기 함수

OpenCV는 영상에서 선, 도형, 문자열을 출력하는 그리기 함수를 제공한다.

선 그리기(직선, 화살표, 마커), 도형 그리기(사각형, 원, 타원, 다각형), 문자열 출력 함수를 살펴보자.

그리기 함수 사용시 주의할 점

- 🔄 영상의 픽셀 값 자체를 변경한다.
- 🔄 원본 영상이 필요하면 복사본을 만들어서 그리기 & 출력을 해야 한다.
- 🔄 그레이스케일 영상에는 컬러로 그리기가 안 된다.
- 🔄 cv2.cvtColor() 함수로 BGR 컬러 영상으로 변환한 후 그리기 함수를 호출한다.

① cv2.line() : 직선 그리기

cv2.line() 함수를 이용하면 영상에 직선을 그릴 수 있다.

함수 설명

```
cv2.line(img, pt1, pt2, color, thickness=None, lineType=None, shift=None) -> img
```

img : 그림을 그릴 영상

pt1, pt2 : 직선의 시작점과 끝점. (x, y) 튜플

color : 선 색상 또는 밝기. (R, G, B) 튜플 또는 정수값

thickness : 선 두께. 기본값은 1

lineType : 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택

shift : 그리기 좌표 값의 축소 비율. 기본값은 0

② cv2.rectangle() : 사각형 그리기

cv2.rectangle() 함수를 이용하면 영상에 사각형을 그릴 수 있다.

함수 설명

```
cv2.rectangle(img, pt1, pt2, color, thickness=None, lineType=None, shift=None) -> img
```

```
cv2.rectangle(img, rec, color, thickness=None, lineType=None, shift=None) -> img
```

사각형은 두 가지 방법으로 그릴 수 있습니다.

- (1) 사각형의 두 꼭지점 좌표 설정하기
- (2) 사각형 위치 정보 (시작점, 높이,길이)

img : 그림을 그릴 영상

pt1, pt2 : 사각형의 두 꼭지점 좌표. (x, y) 튜플

rec : 사각형 위치 정보. (x, y, w, h) 튜플

color : 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값

thickness : 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.

shift : 그리기 좌표 값의 축소 비율. 기본값은 0

③ cv2.circle() : 원 그리기

cv2.circle() 함수를 이용하면 영상에 원을 그릴 수 있다.

함수 설명

```
cv2.circle(img, center, radius, color, thickness=None, lineType=None, shift=None) -> img
```

img : 그림을 그릴 영상

center : 원의 중심 좌표. (x, y) 튜플

radius : 원의 반지름

color : 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.

thickness : 선 두께. 기본값은 1, 음수(-1)를 지정하면 내부를 채움

lineType : 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택

shift : 그리기 좌표 값의 축소 비율. 기본값은 0

④ cv2.polylines() : 다각형 그리기

cv2.polylines() 함수를 이용하면 영상에 다각형을 그릴 수 있다.

함수 설명

```
cv2.polylines(img, pts, isClosed, color, thickness=None, lineType=None, shift=None) -> img
```

img : 그림을 그릴 영상

pts : 다각형 외각 점들의 좌표 배열. numpy.ndarray의 리스트, ex) [np.array([[10, 10], [50, 50], [10, 50]]), dtype=np.int32]

isClosed : 폐곡선 여부. True 또는 False 지정.

color : 선 색상 또는 밝기. (B,G,R) 튜플 또는 정수값

thickness : 선 두께. 기본값은 1, 음수(-1)를 지정하면 내부를 채움

lineType : 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택

shift : 그리기 좌표 값의 축소 비율. 기본값은 0

⑤ cv2.putText() : 문자열 출력

cv2.putText() 함수를 이용하면 영상에 문자열 출력을 할 수 있다.

함수 설명

```
cv2.putText(img, text, org, fontFace, fontScale, color, thickness=None, lineType=None, bottomLeftOrigin=None) -> img
```

img : 그림을 그릴 영상

text : 출력할 문자열

org : 영상에서 문자열을 출력할 위치의 좌측 하단 좌표. (x, y) 튜플

fontFace : 폰트 종류. cv2.FONT_HERSHEY_ 로 시작하는 상수 중 선택

fontScale : 폰트 크기 확대/축소 비율

color : 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.

thickness : 선 두께. 기본값은 1, 음수(-1)를 지정하면 내부를 채움

lineType : 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택

bottomLeftOrigin : True이면 영상 좌측 하단을 원점으로 간주. 기본값은 False

이미지 변경

5.imageTrans.ipynb : 이미지 변경(이동, 회전, 크기 재조정, 대칭)

OpenCV를 이용해 아래의 기능을 수행해 보자.

이미지를 **이동** 시킨다.

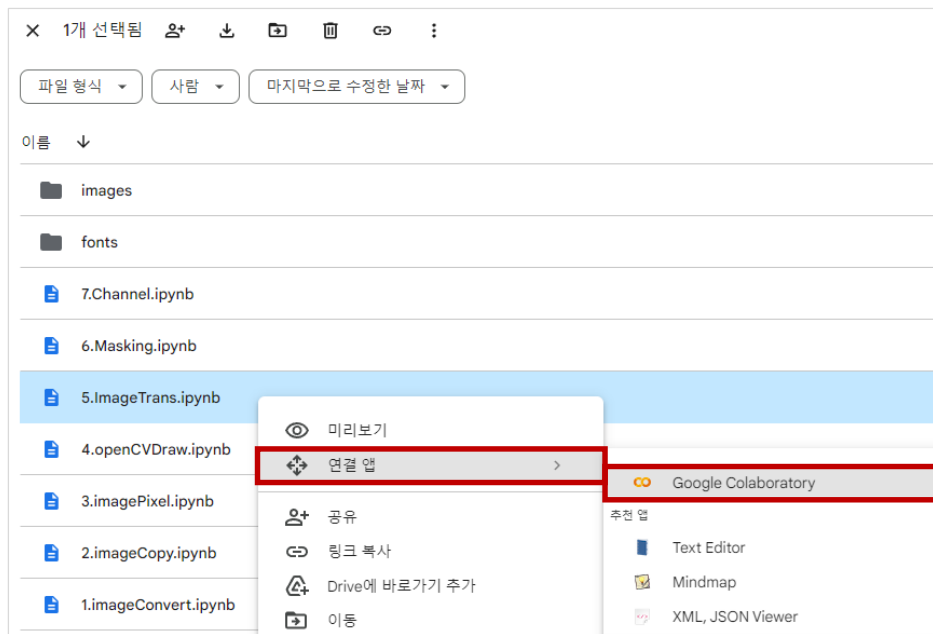
이미지를 **회전** 시킨다.

이미지의 **크기**를 **재조정** 한다.

이미지를 **대칭** 시킨다.

1 ImageTrans.ipynb 구글 Colab 연동

- 아래의 그림의 Basic Vision Skills 폴더에서 5.ImageTrans.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭한다.



1 ImageTrans.ipynb 구글 Colab 연동

■ 아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



2 OpenCV 버전 확인

■ openCV를 사용하기 위해 필요한 모듈을 읽고, openCV 버전을 확인해 보자.

🔄 openCV를 사용하기 위해 **cv2 패키지**를 읽어온다.

```
1 # 필요한 모듈 읽어오기
2
3 import cv2
4 from google.colab.patches import cv2_imshow
```

🔄 openCV를 설치 **버전**을 **확인**한다.

```
1 # OpenCV 버전을 확인합니다
2
3 print("OpenCV version:")
4 print(cv2.__version__)
```

```
➤ OpenCV version:
4.7.0
```

3 구글 Colab 화면에서 내 구글 드라이브 연동

■ 내 구글 드라이브에 연동한다.

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



- [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동된다.
- gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.

3 구글 Colab 화면에서 내 구글 드라이브 연동

[Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용]

아래 화면을 참고하여 gdrive에 연동한다.

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니오 **Google Drive에 연결**

Google 계정으로 로그인

계정 선택

Google Drive for desktop(으)로 이동

장원중
wjang2040@gmail.com

다른 계정 사용

계속 진행하기 위해 Google에서 내 이름, 이메일 주소, 언어 환경 설정, 프로필 사진을 Google Drive for desktop(과/와) 공유합니다. 앱을 사용하기 전에 Google Drive for desktop의 개인정보처리방침 및 서비스 약관을 검토하세요.

한국어 | 도움말 | 개인정보처리방침 | 약관

구글 드라이브 접근 허용

Google 계정으로 로그인

Google Drive for desktop에서 내 Google 계정에 액세스하려고 합니다

wjang2040@gmail.com

어떻게 하면 Google Drive for desktop에서 내 작업을 할 수 있습니까?

- Google Drive 파일 보기, 수정, 생성, 삭제
- Google 포토의 사진, 동영상, 앨범을 봅니다.
- 모바일 클라이언트 구성 및 실행 정보 가져오기
- 프로필 및 연락처와 같은 Google 사용자 정보 조회
- Google Drive 파일 작업 기록 조회
- Google Drive 문서 보기, 수정, 생성, 삭제

Google Drive for desktop 앱을 신뢰할 수 있는지 확인

(간단한 정보가 이 사이트 또는 앱과 공유될 수 있습니다. 언제든지 Google 계정에서 액세스 권한을 확인하고 삭제할 수 있습니다.)

Google이 데이터를 안전하게 공유하는 방법을 알아보세요.

Google Drive for desktop의 개인정보처리방침 및 서비스 약관을 확인하세요.

취소 **허용**

4 원본 이미지 읽기 → 형상(세로, 가로, 채널)

■ 다음은 cv2.imread() 함수로 원본 이미지를 읽어온다.

🔄 원본 이미지를 읽어올 때 컬러로 읽어온다.

```

1  # 이미지를 읽어 온다.
2
3  img = cv2.imread('gdrive/My Drive/CV/Basic Vision Skills/images/rear_garden.PNG', cv2.IMREAD_COLOR)
4  print(img.shape)                # 형상 - (496, 819, 3)
5  print("width: {} pixels".format(img.shape[1]))    # 넓이
6  print("height: {} pixels".format(img.shape[0]))   # 높이
7  print("channels: {}".format(img.shape[2]))        # 채널
    
```

```

📄 (496, 819, 3)
width: 819 pixels
height: 496 pixels
channels: 3
    
```

위 결과에서 원본 이미지의 형상은 (496, 819, 3)인 것을 알 수 있다.

5 원본 이미지 넓이와 높이 확인

■ 다음은 원본 이미지의 높이와 넓이를 변수에 저장하고, 화면에 출력한다.

🔄 이미지의 높이와 넓이는 변수 height, width에 저장한다.

```
1 # 이미지의 높이와 넓이를 변수에 저장, 원본 이미지 출력
2
3 (height, width) = img.shape[:2]
4 print("width: {} pixels".format(width))      # 넓이
5 print("height: {} pixels".format(height))    # 높이
6 cv2_imshow(img)                             # 이미지 출력
```

```
width: 819 pixels
height: 496 pixels
```

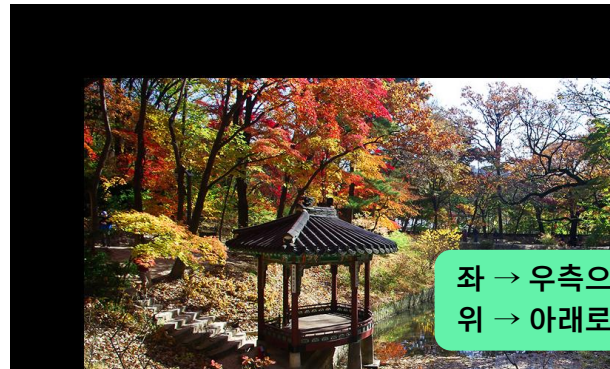


6 원본 이미지 우측 & 위 아래로 100 픽셀 이동

- 다음은 원본 이미지를 좌측 → 우측으로 100픽셀 이동, 위 → 아래로 100픽셀 이동시키고 화면에 출력한다.

🔄 cv2.warpAffine() 함수는 영상의 기하학적 변환, 이동 변환을 수행한다.

```
1 # 이미지를 이동시킵니다. ("Moved right: +, left - and down: +, up: -")
2
3 # 어파인 변환 행렬 생성
4 move = np.float32([[1, 0, 100], [0, 1, 100]]) # [1, 0, 100] → [좌/우, 크기], [0, 1, 100] → [위/아래, 크기]
5 moved = cv2.warpAffine(img, move, (width, height)) # cv2.warpAffine() : 이미지를 이동시킨다.
6 cv2.imshow(moved) # 이미지 출력
```



좌 → 우측으로 100 픽셀 이동
위 → 아래로 100 픽셀 이동

7 이미지 중앙을 기준으로 시계방향 90도 회전

다음은 원본 이미지 중앙을 기준으로 시계방향으로 90도 회전시켜서 화면에 출력한다.

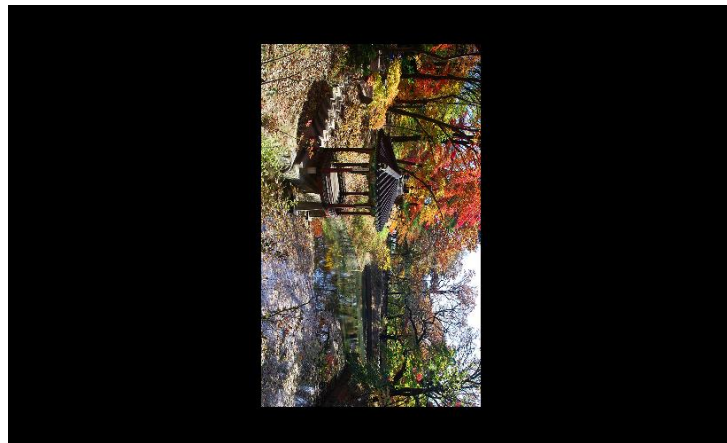
cv2.getRotationMatrix2D() 함수는 이미지를 회전시킨다.

```
1 # OpenCV에서 제공하는 cv2.getRotationMatrix2D 함수를 이용하면 영상의 중앙을 기준으로 회전할 수 있다.
2
3 # 이미지의 중심을 구합니다.
4 center = (width // 2, height // 2)
5 print("image center: {} pixels".format(center)) # 409, 248) - 이미지 중심 픽셀
6
7 # 이미지를 회전시킵니다.
8 rotate = cv2.getRotationMatrix2D(center, -90, 0.5) # (센터, 각도(+:반시계, -:시계), 스케일(확대))
9 rotated = cv2.warpAffine(img, rotate, (width, height))
10
11 cv2.imshow("rotated") # 회전된 이미지 출력
```

원본 이미지



시계 방향으로 90도 회전, 이미지 크기 50% 축소





8 원본 영상 좌측 상단을 기준으로 회전

다음은 원본 **이미지**의 **좌측 상단**을 **기준**으로 **반시계 방향으로 회전**시켜서 화면에 출력한다.

- 영상의 좌측 상단을 기준으로 세타(θ)만큼 회전시킨다.

```
1 # 영상의 좌측 상단을 기준으로 반시계 방향으로 세타만큼 회전시킬 때 sin, cos 함수로 표현할 수 있다.  
2 # np.array로 어파인 행렬을 생성하고, warpAffine() 함수를 이용하여 회전한다.  
3  
4 import math  
5  
6 rad = 10 * math.pi / 90 # 각도 설정  
7  
8 # np.array로 Affine 행렬 생성  
9 aff = np.array([[math.cos(rad), math.sin(rad), 0], [-math.sin(rad), math.cos(rad), 0]], dtype=np.float32)  
10 rotated2 = cv2.warpAffine(img, aff, (0, 0)) # (0,0) 의미는 입력영상 크기와 동일  
11  
12 cv2.imshow(rotated2) # 회전시킨 이미지 출력
```

원본 이미지



반시계 방향으로 세타(θ) 회전



9 이미지 넓이 400픽셀, 줄어든 비율 계산 → 재조정

- 다음은 원본 이미지의 넓이를 400 픽셀로 고정하고, 이때 원본 이미지에서 줄어든 비율을 계산한다.
- ↻ 원본 이미지의 높이에 다시 이 비율을 곱해 축소된 높이를 계산한다.

```
1 # 이미지 크기를 비율에 맞게 재조정 합니다.
2
3 ratio = 400.0 / width
4 dimension = (400, int(height * ratio))
5 print("ratio: {} pixels".format(ratio))          # ratio: 0.4884004884004884 pixels
6 print("dimension: {} pixels".format(dimension))  # dimension: (400, 242) pixels
7
8 # 축소된 넓이, 높이의 사이즈로 이미지를 재조정 한다.
9 resized = cv2.resize(img, dimension, interpolation = cv2.INTER_AREA) # cv2.INTER_AREA - 영상 축소 시 효과적
10 print("resized: {} pixels".format(resized.shape)) # resized: (242, 400, 3) pixels
11
12 cv2.imshow(resized) # 축소된 이미지 출력
```

이미지 넓이 400 픽셀 고정

비율 = 0.4884

변경 height = 496 * 0.4884 = 242.2464

원본 이미지 형상 (496, 819, 3)

변경 이미지 (242, 400, 3)



cv.flip(): 이미지 좌우 대칭

다음은 원본 이미지를 **좌우 대칭**으로 변경시켜 보자.

cv2.flip() 함수는 **이미지 좌우 대칭**을 수행한다.

```
1 # cv2.flip(src, flipCode, dst=None)
2 #   - flipCode : 양수(+1) - 좌우 대칭, 0 - 상하 대칭, 음수(-1) - 좌우 & 상하 대칭
3
4 flipped = cv2.flip(img, 1) # 1 : 좌우 대칭
5 cv2.imshow(flipped)      # 좌우 대칭 이미지 출력
```

원본 이미지



좌우 대칭된 이미지



cv.flip(): 이미지 좌우 & 상하 대칭

다음은 원본 이미지를 **좌우 & 상하 대칭**으로 변경시켜 보자.
cv2.flip() 함수를 이용해 **이미지 좌우 & 상하 대칭**을 수행해 보자.

```
1 # cv2.flip(src, flipCode, dst=None)
2 # - flipCode : 양수(+1) - 좌우 대칭, 0 - 상하 대칭, 음수(-1) - 좌우 & 상하 대칭
3
4 flipped = cv2.flip(img, -1) # -1 : 좌우 & 상하 대칭
5 cv2.imshow(flipped)        # 좌우 & 상하 대칭 이미지 출력
```

원본 이미지



좌우 & 상하 대칭된 이미지



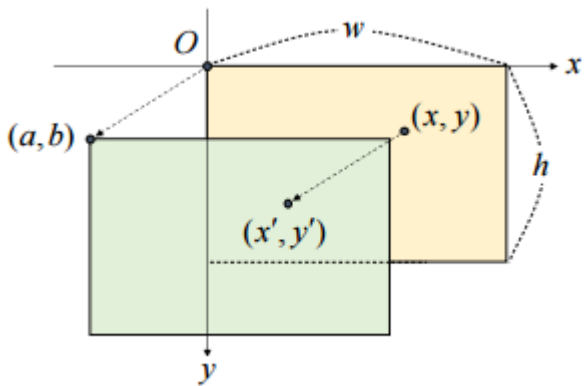
① [참고] 영상의 기하학적 변환, 이동 변화 : `cv2.warpAffine()`

`cv2.warpAffine()`: 영상의 기하학적 변환, 이동

영상의 기하학적 변환(geometric transformation) 이란?

- 영상을 구성하는 픽셀의 배치 구조를 변경함으로써 전체 영상의 모양을 바꾸는 작업이다.
- 영상의 모양 자체를 변환하려면 좌표에 대한 개념이 필요하다.
- 기하학적 변환이 필요한 이유는 다음과 같다.
 - 1 입력 영상 크기가 제한되어 있어 영상 크기를 축소해야 하는 경우
 - 2 객체가 영상의 중심에 있어야 하는 경우
 - 3 회전이 되어 있는 영상을 똑바로 보정하는 경우
- 이 외에도 여러 가지 상황에서 영상의 기하학적 변환이 필요하다.

영상의 이동 변환(translation transformation)



- 이동 변환은 Shift라는 용어도 많이 사용된다.
- 가로 또는 세로 방향으로 영상을 특정 크기만큼 이동시키는 변환이다.
- x, y 방향으로 어느 정도 이동했는지에 대한 변위를 지정해줘야 한다.

$$\begin{cases} x' = x + a \\ y' = y + b \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} a \\ b \end{bmatrix}$$

- 원점에서 x축 이동은 a, y축 이동은 b로 표시한다.

영상의 이동 변환(translation transformation)

- 아래의 좌측 식은 **덧셈 행렬**로 구성되어 있는데, **곱셈 행렬** 형태로 **변환**하여 하나의 수식으로 표현 할 수 있다.

$$\begin{cases} x' = x + a \\ y' = y + b \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} a \\ b \end{bmatrix} \quad \Rightarrow \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2x3 어파인 변환 행렬

- 어파인 변환 행렬**(동차적 표계 개념)을 적용하면 **이동 변환**을 하나의 **행렬**로 **표현**할 수 있다.
- 이동 변환을 표현하는 **어파인 변환 행렬**을 **np.array**로 만들고, cv2.warpAffine() 함수로 넘겨주면 **이동 변환**할 수 있다.

```
# 이미지를 이동시킵니다. ("Moved right: +, left - and down: +, up: -")

# 어파인 변환 행렬 생성
move = np.float32([[1, 0, 100], [0, 1, 100]]) # [1, 0, 100] -> [좌/우, 크기], [0, 1, 100] -> [위/아래, 크기]

# cv2.warpAffine() : 이미지를 이동시킨다.
moved = cv2.warpAffine(img, move, (width, height))
cv2.imshow(moved)
```

① [참고] 영상의 기하학적 변환, 이동 변화 : `cv2.warpAffine()`

```
# 이미지를 이동시킵니다. ("Moved right: +, left - and down: +, up: -")
```

```
# 어파인 변환 행렬 생성
```

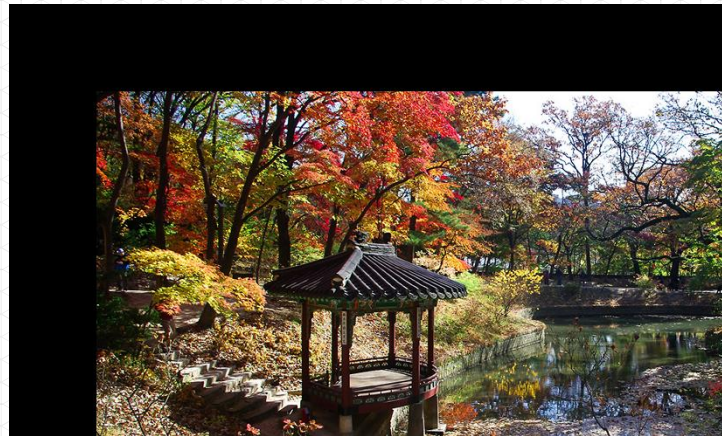
```
move = np.float32([[1, 0, 100], [0, 1, 100]])
```

```
# [1, 0, 100] -> [좌/우, 크기], [0, 1, 100] -> [위/아래, 크기]
```

```
# cv2.warpAffine() : 이미지를 이동시킨다.
```

```
moved = cv2.warpAffine(img, move, (width, height))
```

```
cv2.imshow(moved)
```



① [참고] 영상의 기하학적 변환, 이동 변화 : `cv2.warpAffine()`

영상의 어파인 변환 함수 : `cv2.warpAffine()`

```
cv2.warpAffine(src, M, dsize, dst=None, flags=None, borderMode=None,
borderValue=None) -> dst
```

- src: 입력 영상
- M: 2x3 어파인 변환 행렬. 실수형.
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 src와 같은 크기로 설정.
- dst: 출력 영상
- flags: 보간법. 기본값은 `cv2.INTER_LINEAR`.
- borderMode: 가장자리 픽셀 확장 방식. 기본값은 `cv2.BORDER_CONSTANT`.
- borderValue: `cv2.BORDER_CONSTANT`일 때 사용할 상수 값. 기본값은 0(검정색).

- 🔄 `cv2.INTER_LINEAR`은 **양선형 보간법**을 의미한다.
- 🔄 `borderValue`는 이동 변환을 했을 때 생기는 **빈 공간**을 **어떤 색깔로 채울 것인지를** 의미한다.

② [참고] 영상의 회전 변환

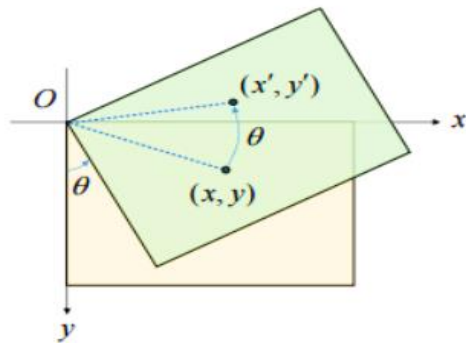
(참고) 영상의 회전 변환

영상의 회전 변환(Rotation transformation)

- 반시계 방향으로 세타(θ)만큼 회전시킬 때 $\sin$, $\cos$ 함수로 표현할 수 있다.
- 어파인(affine) 행렬을 생성하고 `warpAffine()` 함수를 이용하여 회전할 수 있다.
- 회전 변환을 위한 어파인(affine) 행렬을 생성하는 방법은 두 가지가 있다.

$$\begin{cases} x' = \cos \theta \cdot x + \sin \theta \cdot y \\ y' = -\sin \theta \cdot x + \cos \theta \cdot y \end{cases}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



1 영상의 좌측 상단을 기준으로 회전하는 방법

- np.array로 어파인 행렬을 생성한다.
- 생성한 어파인 행렬을 warpAffine() 함수의 입력 인자로 입력해 준다.

```
# 반시계 방향으로 세타만큼 회전시킬 때 sin, cos 함수로 표현할 수 있다.  
  
# 어파인 행렬을 생성하고 warpAffine() 함수를 이용하여 회전할 수 있다.  
  
import math  
  
rad = 10 * math.pi / -90      # 각도 설정  
  
# np.array로 Affine 행렬 생성  
aff = np.array([[math.cos(rad), math.sin(rad), 0],  
                [-math.sin(rad), math.cos(rad), 0]], dtype=np.float32)  
  
rotated2 = cv2.warpAffine(img, aff, (0, 0))    # (0,0) 의미는 입력영상 크기와 동일
```

② [참고] 영상의 회전 변환 : `cv2.warpAffine()`

반시계 방향으로 세타만큼 회전시킬 때 `sin`, `cos` 함수로 표현할 수 있다.

어파인 행렬을 생성하고 `warpAffine()` 함수를 이용하여 회전할 수 있다.

```
import math
```

```
rad = 10 * math.pi / -90      # 각도 설정
```

`np.array`로 Affine 행렬 생성

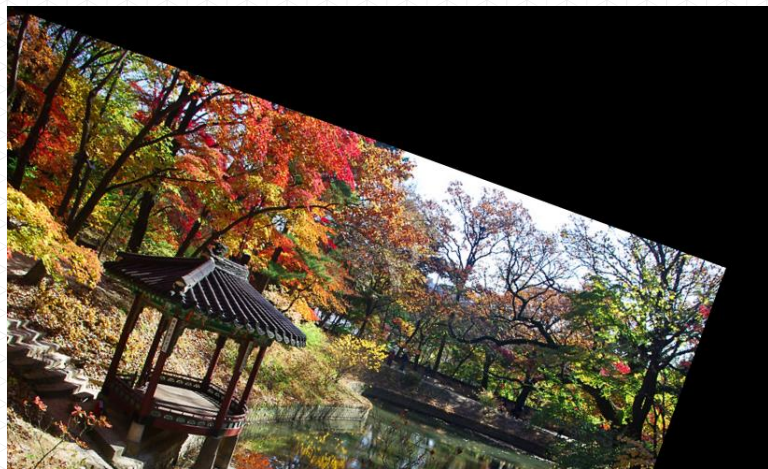
```
aff = np.array([[math.cos(rad), math.sin(rad), 0],  
               [-math.sin(rad), math.cos(rad), 0]], dtype=np.float32)
```

```
rotated2 = cv2.warpAffine(img, aff, (0, 0))    # (0,0) 의미는 입력영상 크기와 동일
```



② [참고] 영상의 회전 변환 : `cv2.warpAffine()`

- 좌측 상단을 기준으로 영상이 회전된 것을 볼 수 있다.
- 출력 영상의 크기를 입력 영상의 크기와 동일하게 지정하여 영상을 아래로 이동한 것을 볼 수 있다.



2 영상의 중앙 지점을 기준으로 회전하는 방법 : cv2.getRotationMatrix2D()

- OpenCV에서 제공하는 cv2.getRotationMatrix2D() 함수를 이용하면 **영상의 중앙**을 기준으로 회전할 수 있다.
- 함수 설명은 다음과 같다.

cv2.getRotationMatrix2D(center, angle, scale) -> retval

- center: 회전 중심 좌표. (x, y) 튜플.
- angle: (반시계 방향) 회전 각도(degree). 음수는 시계 방향.
- scale: 추가적인 확대 비율
- retval: 2x3 어파인 변환 행렬. 실수형.

- 회전 중심 좌표는 영상의 가로, 세로 ½ 값을 넣어주면 **영상의 중앙 좌표**로 **설정**할 수 있다.
- cv2.getRotationMatrix2D() 함수는 결과 값을 2x3 어파인 변환 행렬을 반환한다.

$$\begin{bmatrix} \alpha & \beta & (1 - \alpha) \cdot \text{center.x} - \beta \cdot \text{center.y} \\ -\beta & \alpha & \beta \cdot \text{center.x} + (1 - \alpha) \cdot \text{center.y} \end{bmatrix} \text{ where } \begin{cases} \alpha = \text{scale} \cdot \cos(\text{angle}) \\ \beta = \text{scale} \cdot \sin(\text{angle}) \end{cases}$$

- 이미지의 높이와 넓이를 가져온다.

```
# 이미지의 높이와 넓이를 가져옵니다.  
(height, width) = img.shape[:2]  
print("width: {} pixels".format(width))      # 넓이  
print("height: {} pixels".format(height))    # 높이
```

```
width: 819 pixels  
height: 496 pixels
```

- 이미지 중심을 구한다.

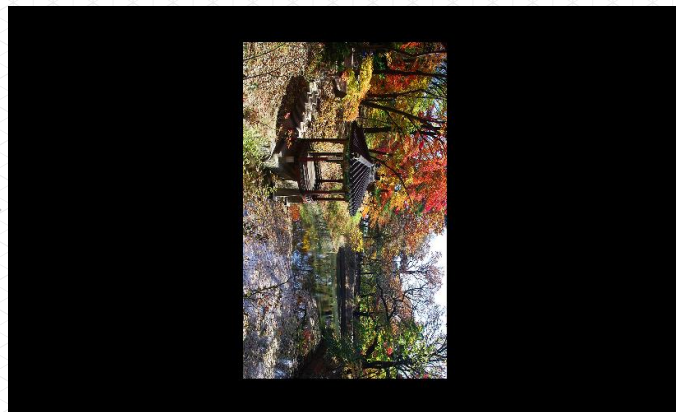
```
# 이미지의 중심을 구합니다.  
center = (width // 2, height // 2)  
print("center width: {} pixels".format(center[0]))    # 넓이  
print("center height: {} pixels".format(center[1]))  # 높이
```

```
center width: 409 pixels  
center height: 248 pixels
```

② [참고] 영상의 회전 변환 : `cv2.getRotationMatrix2D()`

- 이미지를 시계방향으로 90도 회전시키고, 크기는 50%로 축소하여 화면에 출력한다.

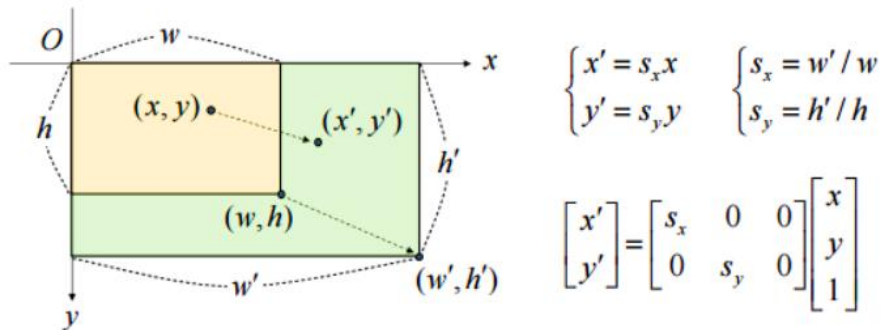
```
# OpenCV에서 제공하는 cv2.getRotationMatrix2D 함수를 이용하면 영상의 중앙을 기준으로 회전할 수 있다.  
  
# 이미지를 회전시킵니다.  
rotate = cv2.getRotationMatrix2D(center, -90, 0.5)      # 센터, 각도(+:반시계, -:시계), 스케일(확대)  
rotated = cv2.warpAffine(img, rotate, (width, height))  
  
# 이미지를 Display 합니다.  
cv2.imshow(rotated)
```



③ [참고] 영상의 확대와 축소 : cv2.resize()

(참고) 영상의 확대와 축소 : cv2.resize()

이미지 크기 변환(Scale transformation)



- 영상의 크기를 원본 영상보다 크게 또는 작게 만드는 변환이다.
- x축과 y축 방향으로 스케일 비율(scale factor)를 지정한다.
- 크기 변환은 빈번하게 사용되는 작업이므로 OpenCV에서는 cv2.resize() 함수를 제공한다.

③ [참고] 영상의 확대와 축소 : cv2.resize()

- cv2.resize() 함수의 설명은 다음과 같다.

```
cv2.resize(src, dsize, dst=None, fx=None, fy=None, interpolation=None) -> dst
```

- src: 입력 영상
 - dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 fx와 fy 값을 이용하여 결정.
 - dst: 출력 영상
 - fx, fy: x와 y방향 스케일 비율(scale factor). (dsize 값이 0일 때 유효)
 - interpolation: 보간법 지정. 기본값은 cv2.INTER_LINEAR
-
- resize() 함수는 출력 영상 크기를 픽셀 단위로 설정할 수 있다.
 - 인자 dsize가 (0, 0)이면, 스케일 비율로 출력 영상을 결정할 수 있다.
 - 그러므로 dsize나 fx, fy 둘 중 하나는 꼭 명시해야 한다.

③ [참고] 영상의 확대와 축소 : cv2.resize()

```
cv2.resize(src, dsize, dst=None, fx=None, fy=None, interpolation=None) -> dst
```

- src: 입력 영상
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 fx와 fy 값을 이용하여 결정.
- dst: 출력 영상
- fx, fy: x와 y방향 스케일 비율(scale factor). (dsize 값이 0일 때 유효)
- interpolation: 보간법 지정. 기본값은 cv2.INTER_LINEAR

- interpolation 인자를 잘 설정하는 것이 중요하다.

cv2.INTER_NEAREST	최근방 이웃 보간법
cv2.INTER_LINEAR	양선형 보간법(2x2 이웃 픽셀 참조)
cv2.INTER_CUBIC	3차회선 보간법(4x4 이웃 픽셀 참조)
cv2.INTER_LANCZOS4	Lanczos 보간법(8x8 이웃 픽셀 참조)
cv2.INTER_AREA	영상 축소 시 효과적

③ [참고] 영상의 확대와 축소 : cv2.resize()

cv2.INTER_NEAREST	최근방 이웃 보간법
	가장 빠르지만 퀄리티가 많이 떨어진다고(따라서 잘 쓰이지 않는다).
cv2.INTER_LINEAR	양선형 보간법(2x2 이웃 픽셀 참조)
	효율성이 가장 좋다. 속도도 빠르고 퀄리티도 적당하다.
cv2.INTER_CUBIC	3회선 보간법(4x4 이웃 픽셀 참조)
	16개의 픽셀을 이용한다. cv2.INTER_LINEAR 보다 느리지만 퀄리티는 더 좋다.
cv2.INTER_LEANCZOS4	Lanczos 보간법(8x8 이웃 픽셀 참조)
	64개의 픽셀을 이용한다. 좀 더 복잡해서 오래 걸리지만 퀄리티는 좋다.
cv2.INTER_AREA	영상 축소 시 효과적
	영역적인 정보를 추출해서 결과 영상을 셋팅한다. 영상을 축소할 때 이용한다.

④ [참고] 영상의 대칭 변환 : cv2.flip()

영상의 확대와 축소 : cv2.resize()

- 영상의 대칭 변환은 크기 변환과 이동 변환으로 구현할 수 있다.
- OpenCV에서는 내부적으로 크기 변환과 이동 변환을 거쳐서 대칭 변환을 구현하는 함수를 제공한다.
- cv2.flip() 함수 설명은 다음과 같다.

```
cv2.flip(src, flipCode, dst=None) -> dst
```

- src: 입력 영상
- flipCode: 대칭 방향 지정
- dst: 출력 영상

⊙ flipCode 인자에는 3가지 입력값을 가진다.

양수 (+1)	좌우 대칭
0	상하 대칭
음수 (-1)	좌우 & 상하 대칭

④ [참고] 영상의 대칭 변환 : cv2.flip()

- cv2.flip() 함수를 이용해 이미지 좌우 & 상하 대칭을 수행해 보자.

```
# cv2.flip(src, flipCode, dst=None)
#   - flipCode : 양수(+1) - 좌우 대칭, 0 - 상하 대칭, 음수(-1) - 좌우 & 상하 대칭

flipped = cv2.flip(img, -1) # -1 : 좌우 & 상하 대칭

# 이미지 출력
cv2.imshow('flipped')
```



위 오른쪽 그림에서 이미지가 좌우 & 상하로 대칭된 것을 확인할 수 있다.